Software Engineering Project

The objective of this project is to practice the Software Engineering concepts and techniques learn in the course. Your team is being asked to build a **software system for managing the operations of a drone delivery company**.

The system should allow users to register customers, create delivery orders, manage packages and drones, maintain drone stations and delivery zones, define and monitor restrictions, schedule deliveries, assign drones, record flights and deliveries, handle exceptions, maintain drone maintenance records, and provide operational reports.

Your team must determine the detailed requirements, business rules, use cases, data structures, system design, and implementation based on the client information provided below.

This is the description of a software system

The DroneDash CLE
Drone Delivery Operations System (CLE_DDOS)

The **DroneDash Cleveland Delivery Company** has asked your software development team to design and implement an application that models and manages a drone-based delivery service in the Cleveland metropolitan area.
The company plans to provide same-day delivery of small packages using autonomous delivery drones. DroneDash operates a fleet of drones from several stations throughout its service area. Customers can request deliveries by providing a pickup location, a destination, a package, a requested delivery time, and a delivery priority.
The goal is to automate customer registration, package orders, delivery scheduling, drone assignment, flight monitoring, delivery completion, exception handling, maintenance tracking, and record-keeping for the company's delivery operations.
Below is an overview of the DroneDash delivery service, as provided by the client.

- **Customers:** DroneDash serves individual and business customers throughout the Cleveland metropolitan area. Each customer has identifying and contact information and may have one or more registered delivery addresses. A customer may place multiple delivery orders. Each order belongs to exactly one customer and identifies a pickup location and a delivery location. Customers

should be able to view the current status of their orders as well as their delivery history.

- **Delivery Orders:** A delivery order represents a request to transport one package from a pickup location to a delivery location. Each order has a unique identification and information such as the customer, package, pickup location, delivery location, requested delivery date and time, priority, and current status. An order may progress through several stages during its lifetime. For example, an order may be Requested, Accepted, Scheduled, Assigned, In Transit, Delivered, Cancelled, or Failed. The company may establish rules determining which status transitions are permitted. The system must prevent invalid transitions.
- **Packages:** Each delivery order contains one package. The system must maintain information about the package, including its identification, weight, dimensions, package type, and any special handling requirements. Packages must satisfy the company's size and weight restrictions. A package that exceeds the permitted capacity of a drone cannot be assigned to that drone. The system must consider applicable package restrictions when determining whether an order can be scheduled.
- **Drones:** DroneDash operates a fleet of delivery drones. Each drone has a unique identification and information such as model, maximum payload, maximum flight range, current location, battery level, and current status. A drone may have statuses such as Available, Assigned, In Flight, Charging, Maintenance, or Out of Service. A drone that is already assigned to an active delivery cannot be assigned to another delivery. A drone must have sufficient payload capacity, available range, and battery level to support a delivery.
- **Drone Stations:** DroneDash operates several drone stations throughout the service area. A station provides a location where drones may begin a delivery, return after completing a delivery, recharge, or undergo maintenance. The system must maintain information about each station and the drones associated with it. A delivery must be assigned to an appropriate station based on the operational requirements of the delivery.
- **Delivery Zones and Restrictions:** The Cleveland service area is divided into delivery zones. Some zones may have restrictions that prevent or limit drone deliveries. A restriction may be permanent or temporary and may be active or inactive. The system must determine whether a delivery can be scheduled based

on its pickup location, destination, and applicable restrictions. A delivery cannot be assigned if its destination is within an active restricted zone. The system must maintain information about delivery zones and their restrictions.

- **Delivery Scheduling:** When an order is accepted, the system determines whether it can be scheduled. Scheduling depends on factors such as package weight and size, drone capacity, drone range, battery level, drone availability, delivery zone restrictions, requested delivery time, and priority. A drone cannot have two overlapping delivery assignments. The system must prevent assignments that violate the operational requirements of the delivery or create scheduling conflicts.
- **Drone Assignment:** When a delivery is scheduled, an appropriate drone must be assigned to the order. The system must verify that the drone is available and that its payload capacity, range, battery level, location, and current status satisfy the requirements of the delivery. A drone undergoing maintenance, charging, or otherwise unavailable cannot be assigned to a delivery. Once a drone is assigned, the system must update its status accordingly.
- **Flight and Delivery:** When a drone begins a delivery, the system records the departure. During the delivery, the system records relevant information about the flight and the current status of the drone and delivery. When the drone reaches the destination, the package is marked as delivered according to the company's delivery procedure. The system records the completion time. If the delivery cannot be completed, the system records the failure and the reason.
- **Delivery Exceptions:** A delivery may encounter problems during scheduling or flight. Examples include insufficient battery, weather restrictions, a restricted delivery zone, drone malfunction, a package problem, or an unavailable destination. An exception may result in the delivery being rescheduled, reassigned to another drone, returned to a station, or cancelled. The system must maintain the history of delivery exceptions and the actions taken in response to them.
- **Rescheduling and Reassignment:** When a delivery cannot proceed as originally planned, DroneDash may need to reschedule the delivery or assign it to another drone. A new drone must satisfy the applicable package, capacity, range, battery, availability, station, and delivery-zone requirements. The system

must ensure that rescheduling or reassignment does not create conflicts with other scheduled deliveries.

- **Drone Maintenance:** Drones periodically require maintenance and may also require maintenance after a malfunction or other operational problem. A drone undergoing maintenance cannot be assigned to a delivery. The system must maintain maintenance records and the current maintenance status of each drone. Once maintenance has been completed, the drone may be returned to an appropriate operational status.
- **Customer Delivery Status:** Customers should be able to obtain information about their delivery orders. The system should provide the current status of an order and relevant delivery information. Customers should also be able to review the history of their completed, cancelled, or failed deliveries.
- **Delivery History:** The system must preserve information about deliveries after they have been completed or have failed. A delivery record should identify the customer, package, pickup location, destination, assigned drone, scheduled delivery information, departure time, completion time when applicable, final status, and relevant exceptions. This information may be used by DroneDash to review its delivery operations.
- **Record-Keeping and Reports:** The system must maintain records of customers, addresses, orders, packages, drones, stations, delivery zones, restrictions, schedules, flights, deliveries, exceptions, and maintenance activities. The system should provide appropriate reports for company administrators. For example, administrators may need to determine current deliveries, available drones, drone assignments, failed deliveries, active restricted zones, drones undergoing maintenance, or the delivery history of a particular customer.

Sample Output
Here is a sample output of the CLE_DDOS

Example Delivery

The following example illustrates the type of delivery that the DroneDash system must be able to model.

Cleveland Medical Supplies requests an Express delivery of package **PKG-88341**. The package is picked up at **Station 2** and must be delivered to **1450 Euclid Ave.**

Before scheduling the delivery, the system determines whether the destination is within an active restricted zone and whether the package satisfies the company's delivery restrictions. The system also determines whether an available drone has sufficient payload capacity, range, and battery level and whether the drone is available during the requested delivery period.

Suppose the system determines that **DR-17** satisfies the applicable requirements. The drone is assigned to the delivery.

The system may record:

Delivery: DD-20481
Customer: Cleveland Medical Supplies
Package: PKG-88341
Pickup: Station 2
Destination: 1450 Euclid Ave.
Priority: Express
Drone: DR-17
Status: Delivered
Departure: 10:34 AM
Arrival: 10:47 AM

The system should maintain the relationships among the **customer, order, package, pickup location, destination, delivery zone, station, drone, schedule, flight, and final delivery record**.

Example Delivery Exception

Suppose that before departure, the assigned drone's battery level becomes insufficient for the planned delivery.

The system may record:

Delivery: DD-20481
Drone: DR-17
Exception: Insufficient Battery
Status: Rescheduling Required

The system must then allow the delivery to be handled according to the company's rules. For example, the delivery may be rescheduled until the drone has sufficient battery, assigned to another available drone, or cancelled.

If another drone is assigned, the system must verify that the new drone satisfies the applicable capacity, range, battery, availability, station, and delivery-zone requirements.

SW & Material

List of SW and material required per group

- If hard copy of the documentation is kept, use a 3-ring binder for the project documentation and bring it to class during meeting
- A white plain soft poster board – append on the wall in class
- Trello for virtual board
- Git&Github for software collaboration
- Any other Software (Lucidchart, StarUML, etc.) of your choice for requirement collection, SW analysis & Design, and SW development.

Sprint 1

Requirements Analysis and Initial Design – Release #1

Duration: 2 weeks

Sprint Goal:
By the end of this sprint, your team should have a clear and agreed-upon understanding of the assigned project, its requirements, users, and major system functionality. You will transform the project description into a Software Requirements Specification (SRS) and begin the design of your system.

Important:
Do **not** begin by writing code. Your first task is to understand the problem and define what the system must accomplish.

1. Analyze the Project Description

Read the assigned project description carefully as a team.

Identify:

- The problem the system is intended to solve.
- The goals of the system.
- The intended users or actors.
- The major functions the system must provide.
- The information/data the system must manage.
- Important constraints or limitations.

- Any assumptions that your team needs to make.

Discuss ambiguities within the team and agree on reasonable interpretations.

Your team should not add unnecessary features simply because they seem interesting. Focus on the functionality required to realize the assigned project.

2. Complete the SRS

Using the **Simplified IEEE SRS Template** provided by the instructor, complete the SRS for your assigned project.

The SRS must be written by your team based on the project description.

For every section:

- Replace the italicized instructions with your team's actual content.
- Do not leave the template instructions in the final document.
- Use clear and precise language.
- Requirements must describe what the system must do, not how you plan to implement it.
- Make sure requirements are consistent with one another.
- Number your requirements so that they can be referenced later.

Functional Requirements

Identify the major functional requirements of the system. Use a bulleted list.

For example:

- The system shall allow a user to create an account.
- The system shall allow an administrator to deactivate an account.
- The system shall notify the user when a specified event occurs.

Your requirements should be specific enough that another team could determine whether the requirement has been implemented correctly.

Non-Functional Requirements

Identify appropriate non-functional requirements, such as:

- Security
- Performance
- Usability
- Reliability

- Availability
- Maintainability
- Privacy

Only include requirements that are relevant to your project.

3. Identify the Actors

Identify all significant actors that interact with your system.

For each actor, briefly describe:

- Who or what the actor is.
- What the actor does with the system.
- What information the actor provides or receives.

Actors may be human users or external systems/services when appropriate.

4. Create the Initial Use-Case Model

Based on your SRS, identify the major use cases of your system.

Create a **Use-Case Diagram** showing:

- Actors
- Major use cases
- Relationships between actors and use cases
- Appropriate include and extend relationships when justified

Do not create relationships simply to make the diagram more complicated. Every relationship should have a clear meaning.

Select the most important use cases for further analysis.

5. Create the Initial Class Model

Identify the major classes/entities that your system will need.

Create an initial **UML Class Diagram** showing, as appropriate:

- Classes
- Important attributes
- Important operations/methods
- Relationships between classes
- Multiplicities
- Inheritance/generalization when appropriate

At this stage, the class diagram is an **initial design**. It will be refined during later sprints as your understanding of the system develops. You can leave relationships for later sprints.

6. Identify Major System Activities

For at least **two important use cases**, create an **Activity Diagram** showing the major workflow.

The diagrams should demonstrate that you understand:

- The sequence of activities
- Decisions and alternative paths
- Conditions
- Parallel activities when appropriate
- The beginning and completion of the process

Choose use cases that involve meaningful system behavior rather than trivial actions.

7. Identify Additional Design Artifacts

Determine whether your project would benefit from additional UML or system-design diagrams.

Examples include:

- Sequence diagrams
- State-machine diagrams
- Component diagrams
- Deployment diagrams
- Entity-relationship diagrams
- System architecture diagrams

You are **not required to create every type of diagram**.

Select diagrams that provide useful information about your particular system and explain briefly why each selected diagram is appropriate.

8. Team Review

Before submitting Sprint 1, the entire team must review the work together.

Verify that:

- The SRS requirements are consistent with the original project description.
- Every major requirement is represented in the system design.
- The use cases correspond to actual system functionality.
- The class diagram supports the identified functionality.
- The activity diagrams represent realistic workflows.
- Different diagrams do not contradict one another.
- Every team member understands the requirements and the proposed design.

Your diagrams and SRS should represent **one coherent system**, not separate pieces produced independently by different team members.

PROGRESS SO FAR: After completing Sprint 1 you have:

- Started becoming familiar with the domain
- Learned the overall requirements of the system
- Captured the major functionalities of the system

Sprint 1 Deliverables - Release #1

Submit **one team submission** containing:

1. Completed SRS
2. Use-Case Diagram
3. Initial UML Class Diagram
4. At least two Activity Diagrams
5. Any additional diagrams your team determines are appropriate
6. Brief Design Decisions section explaining important assumptions or decisions made by the team

All these deliverables will be refined during next Sprints so do not worry if they require further work. Do the best you can at this stage.

Important: No Implementation Yet

Sprint 1 is a requirements and initial design sprint. Do not spend this sprint writing code. Your objective is to establish a sufficiently clear specification and initial design so that development can begin in the following sprint.

The quality of your implementation in later sprints will depend on how carefully you analyze the problem and define the system during this sprint.

Team Responsibility

Every team member is responsible for understanding the **entire SRS and system design**, not only the portion they personally worked on.

You may divide the work among team members, but the final submission must represent a **jointly reviewed and agreed-upon design**.

The objective of this part is to perform ***Requirements Elicitation*** also called ***Requirement Analysis.***

Deadline: 11:59 pm, October 13, 2026. Remember the **late penalties** apply to the whole group.

Sprint 1 Grading Rubric (100 points)

	Criterion	Points	Full-Credit Expectations
1	**SRS Completeness and Organization**	**20**	*All required SRS sections are completed; information is organized, clear, and consistent with the assigned project. No template instructions remain.*
2	**Functional Requirements**	**20**	*Requirements accurately describe what the system must do, are specific and testable, are appropriately numbered, and cover the major functionality in the project description.*
3	**Non-Functional Requirements & Constraints**	**10**	*Relevant quality requirements and constraints are identified and stated clearly. Requirements are appropriate for the particular system rather than generic statements.*
4	**Actors and Use Cases**	**15**	*Major actors and system interactions are correctly identified. The use cases represent the required functionality, and the use-case diagram uses UML notation appropriately.*
5	**Initial Class Diagram**	**15**	*Major classes/entities are identified; important attributes, operations, relationships, and multiplicities are represented where appropriate. The class model supports the requirements and use cases.*
6	**Activity Diagrams**	**10**	*At least two significant use cases are modeled accurately. Activities, decisions, flows, and alternative paths are represented appropriately using UML notation.*
7	**Design Consistency & Decisions**	**5**	*The SRS and diagrams describe the same system and do not contradict one another. Important assumptions/design decisions are documented.*

	Criterion	Points	Full-Credit Expectations
8	**Team Review & Professional Presentation**	**5**	*Submission is organized, readable, professionally formatted, and demonstrates evidence of team review.*

Total: 100 points